Raft and Fault-Tolerant Key/Value Storage

Everton Romanzini Colombo
e257234@dac.unicamp.br

Henrique Ribeiro Medeiros da Silva
h243172@dac.unicamp.br

I. Introduction

Replicating a service across several machines is the standard way to make systems resilient to machine crashes, but copying alone is not enough: the replicas must agree on a single order of operations, and they must keep agreeing while links drop messages, nodes leave and reenter the grid, leaders disappear, among other failures. Many consensus algorithms that provide a solution to this problem exist today, and maintaining a log of agreed-upon state-machine operations is a common implementation approach. The Raft algorithm [1] is a widely used consensus protocol that is designed to be understandable and easy to teach, and it is the basis of many production systems, including etcd, Consul, and CockroachDB. The practical question, however, is not only whether a protocol can be implemented correctly, but what that correctness costs in throughput, tail latency, and recovery time when faults actually happen.

With this in mind, this project had two main elements: 1. Implementing Raft, as well as a simple key-value storage system on top of it, with the goal of understanding in detail how this consensus algorithm works; and 2. Building an experimental harness on top of these implementations to measure the cost of fault tolerance in a realistic environment. By implementing the resulting system, key distributed systems concepts were explored, such as leader election, consensus, replication and consistency, fault detection and tolerance, peer-to-peer communication, among others.

Basis and contributions

This project's starting point was the framework of MIT's 6.824 (2020) labs 2 and 3 [2], which supplied stub interfaces, an in-memory simulated network for tests, the Porcupine linearizability checker [3], and, crucially, a thorough set of tests. The protocol and service implementations, client interface, container packaging, and experimental apparatus were self-written. Appendix B lists exactly what was kept, changed, and written. Concretely, this project's contributions are:

- **In-House Raft Implementation:** following MIT 6.824's labs 2A–2C + 3B, our custom raft implementation closely follows Raft's paper's guidelines, including leader election with randomized timeouts, the log up-to-date voting rule, log replication with fast backup, safety rules for commitment across terms, persistent state restored after a crash, and log compaction with snapshots.
- **Fault-tolerant Key/Value Storage Service Implementation:** following MIT 6.824's labs 3A–3B, a replicated key-value storage service was built on top of the Raft implementation built before. It includes client-side leader discovery with retry, response caching, and state snapshotting.
- **Deployment Setup**: the system's transport layer, ports, and persistence behavior were made configurable via environment variables, so that the same code can run both in the labs' test suites (over the simulated `labrpc` network) and in a containerized deployment (over real `net/rpc` TCP). A Dockerfile for container image configuration, and a `compose.yaml` file for setting up multiple nodes were also written.
- **Interface and Experimental Harness**: an HTTP API was exposed so that clients can interface with the KV application. A `/status` endpoint was also created, exposing raft state variables, plus a benchmark client and an experiments script that simulates multiple scenarios.

The remainder of this report is organized as follows. Section II states the system model and the failure assumptions. Section III details the implemented algorithms. Section IV describes the architecture and the container deployment. Section V argues the correctness properties and describes the automated tests. Section VI defines the experimental methodology and its reproducibility, Section VII presents the results, and Section IX concludes. Since this report's main text assumes the reader is familiar with Raft, Appendix A briefly reviews the protocol.

II. System Model

This section describes the assumptions made when implementing Raft and the KV service on top of it. We considered a fixed set of N servers that talk to each other, plus any number of clients. These server nodes are both KV-store servers and Raft peers. Clients come and go and only send requests. The assumptions listed here are used when arguing correctness and progress in Section V.

a) Timing model: Raft is partially synchronous. It does not know in advance any hard limit on message delay or on how long a node takes to execute a step, but timeouts and heartbeats managed by local timers are used to detect failures and guarantee progress. Safety does not depend on timing, but progress does depend on timeout constants, namely: heartbeats every 150 ms, election timeouts of 500–700 ms, peer RPCs bounded at 800 ms and client requests at 2 s.

b) Channels: Each pair of nodes has its own connection, and messages are addressed to one specific node and there is no broadcast medium. Raft communication happens via remote procedure calls, and client-server interaction with the KV-store

service layer is done with HTTP requests. We assume that a channel can lose, delay, duplicate and reorder messages, and that a message may take arbitrarily long or never arrive. Retransmission and retries are the client's job, not the channel's.

c) Failures: It is assumed that a node can crash and later restart, losing its volatile state but keeping what it wrote to disk. The network can also split into partitions, for a while or forever. We do not consider Byzantine faults; messages are not forged or corrupted. Raft guarantees correctness in (and thus limits progress to) situations in which, with N replicas, no more than $\lfloor (N-1)/2 \rfloor$ of them fail concurrently.

d) Clocks, persistence and restart: Raft does not assume synchronized clocks, and no algorithm step reads wall-clock time; local monotonic clocks are used only to measure timeouts. Before a node replies to a request or acts on one, it writes to disk what it needs after a crash: the current term, the vote, the log and the last included snapshot index. Each write goes to a temporary file that is then renamed, so a crash in the middle of a write cannot leave truncated state. On restart the node reloads those files and resets only its volatile state, coming back as a follower.

III. Implemented Algorithms

The assignment asks for two complementary algorithms, one from the leader election group of topics and one from the consensus and replication group. Raft not only gives us both, but actually goes quite beyond them. Leader election (Section III-A1) picks a single node to order the commands, and log replication (Section III-A2) gets a majority of nodes to agree on that order. On top of both, the key/value service (Section III-B) turns the agreed-upon log into a linearizable store. The produced code mostly follows the specifications of the Raft paper [1], and most of it is in `src/raft/raft.go`.

A. Raft Implementation

Essentially all Raft logic is encapsulated in the `Raft` struct, which holds the replicated log and other state variables. A background goroutine loops over the node's current role and runs the behavior for that role (follower, candidate or leader), a second goroutine delivers committed entries to the service, and every outgoing RPC is sent from its own goroutine without holding the lock, so a slow or unreachable peer never blocks the others. The log is a slice of `(Command, Term)` entries in which position 0 is a placeholder.

*1) **Leader election and terms**:* Each node keeps `currentTerm`, `votedFor`, its role, and the time its election timer was last reset. A follower checks every 50 ms whether 500 ms have passed since that reset, and the reset itself adds a random delay of 0 to 200 ms, so the actual timeout is random between 500 and 700 ms. Having this range is needed to avoid multiple nodes starting elections at the same time. The timer is reset when the node grants a vote, when it receives `AppendEntries` or `InstallSnapshot` from a leader of the current term, and when it starts an election. The leader sends a heartbeat every 150 ms.

When the timer expires, the node becomes a candidate and runs an election. It increments `currentTerm`, votes for itself, and sends `RequestVote` to every other node in parallel, with its term and the index and term of its last log entry. Replies are then collected, and the election ends in one of four ways: a majority grants the vote and the node becomes the leader; a majority refuses; some reply carries a higher term, and the node goes back to being a follower; or the election timeout expires and the node tries again with a new term. The candidate also gives up if its role or term changes in the meantime, for example because a leader with a newer term contacted it.

A node that receives `RequestVote` first applies the term rule: if the candidate's term is higher, it adopts that term, forgets its vote and becomes a follower. It then grants the vote only if it has not voted for someone else in this term and the candidate's log is at least as up to date as its own, meaning the candidate's last entry has a higher term, or the same term and an index at least as high. A granted vote is persisted before the reply is sent. The same term rule is applied to every RPC request and reply in the implementation, so any node that sees a higher term steps down immediately, whatever its role.

A leader keeps track of its followers' log state through two arrays: `nextIndex`, which is the index of the next log entry to be sent to each follower, and `matchIndex`, which is the known last log entry index held by each follower that matches the leader's log. A new leader sends heartbeats right away instead of waiting for the next period, so the other nodes learn about it quickly. Every node also remembers `leaderId`, the last leader it heard from, which is cleared whenever the term changes. The protocol itself does not use it, but it is exposed in `/status`, and that is how the experiment driver finds the leader and measures how long a new election takes (Section VI).

The heartbeats and the timeout together also act as Raft's failure detector. The detector can be wrong, for example when the leader is only slow or its messages are delayed, but a wrong suspicion only costs an extra election, never safety, because the term and voting rules do not depend on timing.

*2) **Log replication and consensus**:* The client service submits a command to the raft layer via the `Start` method. If the node is not the leader, it returns immediately and says so. Otherwise, it appends the command to its log with the current term, saves the log, starts replicating to all followers, and returns the index the entry will have if it is committed.

Replication to each follower is done by a single function, `replicateTo`, which is used both for heartbeats and for new entries. It builds an `AppendEntries` request with `PrevLogIndex` set to `nextIndex`-1, the term of that entry as `PrevLogTerm`, every entry from `nextIndex` to the end of the leader's log, and the leader's `commitIndex`. This means that a heartbeat sent to a follower that is behind is also the message that catches it up. If the follower accepts, the leader updates its `matchIndex` and `nextIndex`; if it refuses, the leader moves `nextIndex` back as described below, and the next call retries from there.

On the follower side, the handler applies the term rule, rejects

requests from older terms, records the sender as the current leader and resets the election timer. It then runs the consistency check: if it has no entry at `PrevLogIndex`, or the entry there has a different term, it refuses, indicating to the leader that it should send even older log entries by decrementing the follower's `nextIndex` value. Otherwise, it walks through the new entries, skips the ones it already has, cuts its log at the first one that conflicts and appends the rest. Finally, it saves the log, sets its `commitIndex` to the smaller of the leader's commit index and its own last index, and wakes up the goroutine that applies entries.

After each reply, the leader looks for the highest index stored on a majority of nodes, itself included, using `matchIndex`. It scans backwards from the end of its log and stops at the first entry that is not from its current term, so it never commits an old entry by counting replicas (the Figure 8 rule). On every node, a separate goroutine waits on a condition variable until `commitIndex` passes `lastApplied`. It then copies the newly committed entries, and sends them to the client service on `applyCh`.

*3) **Fast backup**:* Moving `nextIndex` back by one entry per refused request can take a long time when a follower has a long run of conflicting entries. So a follower that refuses also sends three hints, following the scheme from the 6.824 course [2]: `XTerm`, the term of its conflicting entry; `XIndex`, the first index it holds with that term; and `XLen`, the length of its log. If the follower's log is too short, the leader jumps to `XLen`. If the leader has entries from `XTerm`, it jumps to its own last entry of that term; otherwise, it skips the follower's whole term by jumping to `XIndex`. Either way, the leader moves back by at least a whole term per round trip instead of a single entry.

*4) **Persistence**:* The current term, the vote, the log and the snapshot index are saved on "disk" whenever one of them changes, always before the reply to the RPC that caused the change is sent. On restart, the node reads them back and starts as a follower, essentially picking up from where it stopped.

*5) **Snapshots**:* To avoid letting the replicated log grow indefinitely, the client service decides when to take a snapshot: after applying an entry, if Raft's saved state has reached `maxraftstate` bytes, it serializes its own state and calls `Snapshot(index, data)`. Raft then drops all entries up to `index`, keeps that entry as the new placeholder at position 0, and saves the trimmed state and the snapshot together. When a follower needs entries that the leader has already dropped, `replicateTo` sends `InstallSnapshot` instead, carrying the whole snapshot in a single message. The follower does not install it immediately. It keeps it as a pending snapshot and lets the apply goroutine hand it to the service through `applyCh`, in order with the other messages. The service then calls `CondInstallSnapshot`, which only accepts the snapshot if it is ahead of everything the node has committed. In that case, Raft keeps the part of its log after the snapshot if it agrees with it, discards the whole log otherwise, and moves `commitIndex` and `lastApplied` to the snapshot index.

B. Fault-Tolerant key/value service

After implementing Raft, a fault-tolerant, linearizable, distributed key/value service was built on top of it; most of its code is in `src/kvraft/server.go`. Each node runs a `KVServer` on top of its Raft peer. Its state is the key/value map plus, for each client, the `Seq` and result of the last operation applied for that client, which makes up the response cache. The service supports `Get`, `Put` and `Append`, and every one of them, reads included, is wrapped in an `Op` record with the client's `(ClientId, Seq)` pair and submitted to the Raft layer with `Start`. Sending reads through the log is the simplest way to make them linearizable.

When a request arrives, the handler registers a channel for its `(ClientId, Seq)`, calls `Start`, and records which request it expects at the returned log index. It then waits for up to 5 s. Meanwhile, a single goroutine reads `applyCh` and does the following for every committed command:

1) It checks whether a different request was waiting for this index. If so, the node lost its leadership and another leader's entry took that position, so the waiting request is answered with `ErrWrongLeader`.
2) It compares the command's `Seq` with the cached one for that client. If it is not newer, the command is a duplicate and gets the cached result.
3) Otherwise, it applies the command to the map and records the new `Seq` and result in the cache. If Raft's saved state has reached `maxraftstate`, it also serializes the map and the cache and passes them to `Snapshot`.
4) It wakes up the handler waiting for this request, if there is one on this node.

A request that is not committed within 5 s is answered with `ErrTimeout`. Over HTTP, `ErrWrongLeader` becomes status 421 and `ErrTimeout` becomes 504. When a snapshot arrives through `applyCh`, the server calls `CondInstallSnapshot` and, if Raft accepts it, replaces its map and cache with the ones in the snapshot. On startup, it loads the latest snapshot from its persister, and Raft replays the entries that come after it.

On the client side, the `Clerk` (`src/kvraft/client.go`) picks a random `ClientId` when it is created and increments `Seq` for every new operation. It sends each request to the server that last answered it successfully, and on any failure (no reply, `ErrWrongLeader` or `ErrTimeout`) it moves on to the next server and tries again with the same `Seq`, for as long as it takes.

IV. Architecture

Regarding the dockerized deployment setup, every node runs the same Go binary (`cmd/kvnode`), with one container per node, orchestrated by Docker Compose (Figure 1). Inside each container a single process hosts both the `KVServer` and its `Raft` peer. The two share memory and exchange messages through the `applyCh` channel and `Raft.Start`, while different nodes talk to each other only over the network. Each node therefore opens two listeners: a TCP port for the

internal Raft RPCs and an HTTP port for the client-facing key/value API.

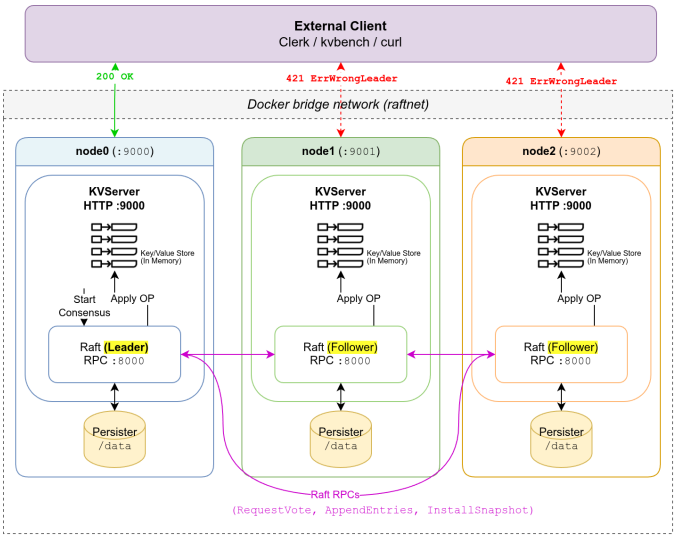


Figure 1. Containerized cluster architecture diagram.

The nodes are configured through environment variables. The `compose.yaml` file on this project's repository contains an example of how to set these variables for a cluster of three nodes.

Clients have their own communication channel, separate from the one used by Raft, via HTTP over the endpoints described in Table I. The `Clerk` in the client library remembers the node it last got an answer from and, if it got back en error, advances to the next server and retries. Followers answer read and write requests with HTTP `421` and `ErrWrongLeader` rather than forwarding them, so leader discovery is entirely client-side. Every request carries a `(ClientId, Seq)` pair so that retried requests are answered from a cache instead of being applied twice.

The HTTP API also exposes a `/status` endpoint that reports the term, the observed leader, the commit, last-log and applied indexes, and the sizes of the Raft state and snapshot; the experiments in Section VI poll it to derive the recovery metrics.

Table I
HTTP ENDPOINTS EXPOSED BY EVERY NODE

Method	Path	Effect
GET	`/status`	node state: term, leader, indexes, state and snapshot sizes
GET	`/kv/{key}`	read the value of a key
PUT	`/kv/{key}`	set the key to the request body
POST	`/kv/{key}`	append the request body to the key

Finally, each node persists its durable state to a mounted volume. The raft side persists the current term, vote and log, and the kvserver side persists the key/value map and the response cache. For more details, see (Section **??**).

V. CORRECTNESS PROPERTIES AND TESTS

The Raft paper [1] already proves that the protocol is safe, so we did not try to prove anything new. What we had to make sure of was that our code really follows the rules that proof depends on, and then test that it does. This section goes through the safety properties this system claims, shows where each one comes from in the implementation, and then describes the tests we used to check them.

A. Safety arguments

a) One leader per term: A node grants at most one vote per term, and it saves `votedFor` and `currentTerm` before replying, so not even a crash makes it forget a vote. A candidate needs votes from more than $N/2$ nodes, its own included. Two leaders in the same term would need two such majorities, and any two majorities of the same N nodes share at least one node, which would have had to vote twice.

b) Log Matching: A leader only appends to its own log, so it creates at most one entry per index in its term. Each `AppendEntries` RPC carries the index and term of the entry right before the new ones, and the follower refuses it unless it has that exact entry. Every accepted request therefore extends a prefix that the follower already shares with the leader, and by induction, if two logs have an entry with the same index and term, they agree on everything before it.

c) Committed entries are never lost: The main idea that supports this is the election restriction, which states that a node only votes for a candidate whose log is at least as up to date as its own, meaning its last entry has a higher term, or the same term and an index at least as high. A committed entry is stored on a majority and the new leader was voted in by a majority, so at least one of a leader's voters has the entry and would have refused a candidate without it. The paper turns this intuition into an induction over terms.

d) Same commands in the same order: `commitIndex` never decreases, and a single goroutine per node delivers entries to the service strictly in index order. Together with the two properties above, this means every replica applies the same command at each index, so all KV replicas go through the same sequence of states.

e) Linearizability: Every client operation, `Get` included, is turned into a log entry, and the server only replies after that entry is applied. The moment of application falls between the request and the reply, which is exactly the instant linearizability asks for. It also means that an old leader that was cut off cannot serve stale reads: it cannot commit anything, so its reads never return. The tricky part is retries. If a reply is lost, the `Clerk` sends the request again, possibly to another server, and the same command can end up in the log twice. Each server stores, for every client, the `Seq` and result of the last operation it applied, and a request whose `Seq` is not newer than that gets the stored result instead of being applied a second time. Since a `Clerk` has only one request in flight at a time, remembering the last one is enough. This table is saved in the snapshot, so it survives restarts and snapshot installs.

f) Liveness: Unlike safety, liveness depends on timing. Randomized election timeouts make repeated split votes unlikely, and once a leader can reach a majority, its heartbeats keep the followers from starting new elections while it retries `AppendEntries` until every entry is replicated.

B. Automated tests

The 6.824 labs' tests run every peer inside a single Go process, connected through `labrpc`, which gives the tester full control of the network. It can disconnect a node, split the cluster into two partitions, or crash a node and later restart it from a copy of what it had persisted. It can also make the network unreliable with artificial delays and packet loss.

The Raft tests (2A–2C) are particularly strong, since safety is checked all the time, not only at the end. The tester records every entry that each peer applies, and fails immediately if two peers apply different commands at the same index, or if a peer applies an index before the previous one. Every time it looks for the leader, it also fails if it sees two leaders in the same term. So every Raft test, whatever it focuses on, is also checking that all replicas apply the same commands and that there is only one leader per term. The KV tests (3A–3B) look at the system from the outside instead, checking the values that clients get back. Every test also has a 120 s limit, so an implementation that stays safe but takes too long to recover also fails. Table II lists the tests by part and how long they took to execute; Appendix C describes each one in detail. In short:

- **2A tests:** focus on leader election. They test whether raft elects exactly one leader, triggers a new election after the leader is disconnected, refuses to elect without a majority, and keeps a returning old leader from disturbing the new one.
- **2B tests:** focus on log replication. They test whether raft commits with followers missing, forces a leader to overwrite long conflicting suffixes after partitions.
- **2C tests:** focus on persistence. They crash and restart nodes in every relevant order, verifying correctness with clients writing while nodes fail and recover.
- **3A tests:** test KV server functionalities by having concurrent clients append to and read their keys under an unreliable network, partitions and restarts.
- **3B tests:** focus on snapshotting. They cap the persisted Raft state and snapshot size, and force a lagging server to catch up through `InstallSnapshot`.
- **3A & 3B linearizability tests:** Porcupine checks the recorded client histories for a valid real-time ordering.
- **Our test:** runs the deployed wiring (real TCP, HTTP, files on disk) through all KV Server operations.

VI. Experimental Methodology

All experiments run on the containerized deployment of Section IV, with every node on the same machine (Table III). A single script, `scripts/run_experiments.py`, runs every configuration of Section VI-C, each on a fresh cluster, and saves the raw results. `scripts/README.md` explains how to run it, and `scripts/analysis.ipynb` turns the results into the tables of Section VII.

Table II
Automated tests and their running time

Part	Focus	Tests	Time (s)
2A	leader election	2	10.1
2B	log replication	8	62.2
2C	persistence, Figure 8, churn	8	129.3
3A	KV service, linearizability	13	249.3
3B	snapshots	8	152.5
Ours	real TCP, HTTP and disk	1	0.7
Total		40	604.1

Table III
Execution environment

Item	Value
Hardware	Intel Core i5-1334U, turbo boost disabled; 7.6 GiB RAM
Operating system	Windows 11, WSL2 (Linux 6.6.87.2)
Docker / Compose	29.9.2 / 5.6.1
Go / Python	1.22.2 / 3.12.3
Nodes	3, 5, 7 containers (same host)
Load	8 clients, 200 ops each
Seeds	10 per configuration
Warm-up	1 discarded run before the fault-free runs with 3, 5 and 7 nodes

A. Workload and scenarios

The load comes from `cmd/kvbench`, a small benchmark client that acts as the users of the key/value service. It runs eight clients, and each of them sends 200 operations, waiting for the reply to one before sending the next. Half of the operations are writes and half are reads, all on the same five keys.

We ran this load in four scenarios, chosen to cover the failures assumed in Section II. The *normal* scenario, where nothing goes wrong, gives us a reference. In the *delay and loss* scenario, `tc netem` delays the packets that each container sends or drops a fraction of them. In the *crash* scenario, we stop the leader's container (`docker stop`), run the load on the remaining nodes once they have elected a new leader, and then restart the old leader and wait for it to catch up. The *partition* scenario follows the same steps, except that the leader is cut off from the network (`docker network disconnect`) instead of stopped, and later reconnected.

In the crash and partition scenarios, we also measure how long the cluster takes to recover from the fault. For that, a separate client, the *probe*, keeps writing from before the fault until the service returns, waiting for each write to finish before sending the next, and logs when each one starts and ends. Meanwhile, the experiment script asks every node for its state through `/status` (Section IV) every 50 ms.

Table IV
THROUGHPUT AND LATENCY (MEDIAN OVER SEEDS)

Scenario	ops/s	p50	p95	p99
Normal	1633	4.5	7.4	8.8
`netem` 50 ms/2%	41	155	307	418
Leader crash (2 nodes)	2038	2.7	8.0	21.9
Partition (after heal)	1227	2.6	6.4	184

B. Metrics

For leader election, the main question is how long the cluster stays without a leader. From the node states we measure the *time to elect*, from the fault until some node reports being leader in a newer term. It should be close to one election timeout (500 to 700 ms, Section III-A1), since a follower waits that long without heartbeats before starting an election, which usually succeeds in its first round. The probe gives us the time to recover service (*TTR-service*): the time from the fault until its first write that completes after it. It can be much longer than the election, since a client whose request went to the old leader only tries another node after its 2 s request timeout (Section II). We also count elections that start while the leader is still running, which happen when its heartbeats arrive too late, for example under delay or loss. The candidate's newer term makes the running leader step down, so each of these elections leaves the cluster without a leader until it ends, even though no node has failed.

For log replication, the main question is what it costs to agree on every operation. This cost shows in the throughput and the 50th, 95th and 99th latency percentiles of each run. Since each client waits for a reply before its next request, the throughput is set by latency, not by how many requests the cluster could handle. After a crash or a partition, the node states also give us the time to recover a replica (*TTR-replica*): how long the returning node takes to apply everything the new leader had committed.

C. Configurations

Each parameter was tested at three levels while the others kept their default value (no delay, no loss, three nodes): a delay of 0, 50 or 100 ms, a loss rate of 0, 2 or 5 %, and clusters of 3, 5 or 7 nodes. The cluster size was varied not only in the normal scenario but also in the crash and partition scenarios. Each of these configurations ran with the same ten seeds, so all of them receive the same sequences of operations, and we report the median over the seeds.

VII. RESULTS

[TODO: Replace the values below with the final version and generate the figures with `scripts/analysis.ipynb`.**]**

A. Normal operation

[TODO: Describe throughput and percentiles; explain that the 2% retry in the baseline is one extra attempt per client to discover the leader (8/400), not steady-state retrying.**]**

Table V
RECOVERY TIMES (MEAN ± STD)

Scenario	election	TTR-service	TTR-replica
Leader crash	507 ± 114 ms	1868 ± 32 ms	468 ± 124 ms
Partition	349 ± 185 ms	1632 ± 404 ms	not applicable

B. Delay and loss

[TODO: Discussion: throughput drop and p99 increase; why the ceiling is dominated by the election timeout and retransmissions; linearizability is preserved.**]**

C. Leader crash

[TODO: Time-to-elect $\approx$ election timeout; TTR-service dominated by the client HTTP timeout ($\sim$2 s); TTR-replica measures recovery via log/snapshot.**]**

D. Partition and the disruptive-server problem

[TODO: Inflated terms, churn, attempts per operation; explain that without PreVote the isolated node exiles itself; treat it as a limitation and future work.**]**

E. Scalability

[TODO: Results with $N = 3, 5, 7$: throughput and latency; discuss why the leader serializes and where replication cost would show up (message counts).**]**

VIII. LIMITATIONS

A new leader does not append the empty no-op entry that Raft prescribes at the start of each term [1], so entries that the previous leader had already replicated but not yet committed stay uncommitted until a client sends a new command (Section III-A2).

We also did not implement PreVote and check-quorum, two common extensions that deal with isolated nodes. Without them, an isolated follower keeps increasing its term and, when it comes back, forces an unnecessary election. An isolated leader keeps accepting requests it can never commit.

The Raft state and the snapshot are saved in one call, but to two separate files. If the node crashes between the two writes, it can end up with a trimmed log next to an older snapshot, silently missing the operations in between.

Finally, all nodes share one host, so they are not independent failure domains, while replication is only effective when nodes fail independently of one another.

IX. CONCLUSION AND FUTURE WORK

[TODO: Recap: two algorithms, correctness under faults, performance cost. Future work: PreVote/check-quorum, batching, lease-based reads, incremental snapshots, message accounting, execution across multiple machines.**]**

APPENDIX A
BACKGROUND: RAFT

A common way to build a fault-tolerant service is the *replicated state machine* [4]: every server keeps its own copy of the service state (in our case, a key/value map) and a log of commands. If all servers apply the same commands in the same order, all copies go through the same states and end up equal, so any of them can answer for the service (Figure 2). The hard part is getting the servers to agree on the contents of the log while messages get lost and servers crash, which is a consensus problem. Raft [1] solves it for a fixed group of N servers, and keeps working as long as a majority of them (more than $N/2$) is up and able to talk to each other. A cluster of three servers survives one failure, and a cluster of five survives two.

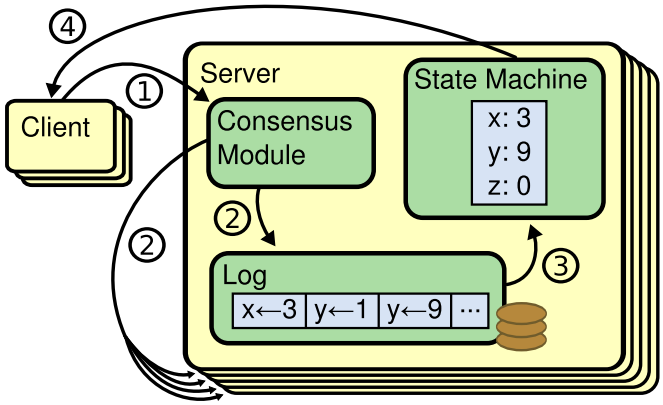


Figure 2. Replicated state machine architecture: the consensus module keeps a replicated log, and each state machine applies the same commands in the same order. Adapted from Figure 1 of [1].

a) Roles and terms: At any moment, each server is a *follower*, a *candidate* or the *leader* (Figure 3). In normal operation there is exactly one leader, and the other servers are followers that only answer its requests. Time is divided into *terms*, numbered with consecutive integers; each term starts with an election and has at most one leader. Every message carries the sender's term, a server that sees a term higher than its own adopts it and becomes a follower, and messages from older terms are rejected. This is how a leader that was cut off finds out that it has been replaced. The basic protocol needs only two RPCs, `RequestVote` and `AppendEntries`. Optionally, a `InstallSnapshot` RPC is defined for log compaction.

b) Leader election: The leader sends periodic heartbeats (`AppendEntries` calls, possibly empty) to every follower. A follower that hears nothing for an *election timeout* assumes the leader has failed: it increments its term, becomes a candidate, votes for itself and asks the other servers for their votes. Each server gives at most one vote per term, to the first candidate that asks, and only if that candidate's log is at least as up to date as its own. A candidate that gets votes from a majority becomes the leader and starts sending heartbeats. If the votes are split and nobody wins, the candidates time out and try again in a new term. Election timeouts are chosen at random, so this shouldn't happen many times in a row.

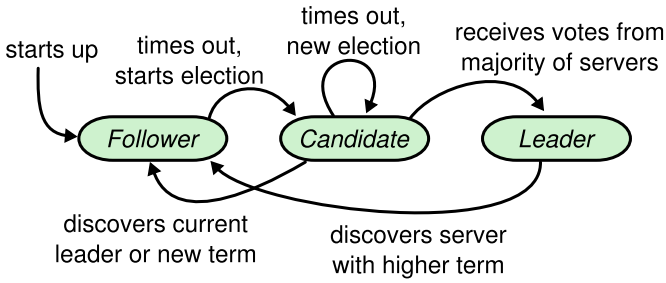


Figure 3. Raft server states and transitions. Adapted from Figure 4 of [1].

c) Log replication: Clients send their commands to the leader, which appends each one to its log as a new entry tagged with the current term, and sends it to the followers with `AppendEntries`. Each call also carries the index and term of the entry just before the new ones, and a follower only accepts it if its own log has a matching entry at that position. If not, the follower refuses, the leader retries from an earlier position, and once the two logs match, the follower's conflicting entries are replaced by the leader's. When an entry is stored on a majority of servers, the leader considers it *committed*, applies it to its state machine and answers the client service. Followers learn the new commit index from later `AppendEntries` calls and apply the entry too.

d) Persistence and snapshots: Each server saves its current term, its vote and its log to stable storage before answering any RPC, so that it can crash, restart and rejoin without breaking promises it made before the crash. Since the log would otherwise grow forever, the service periodically saves a *snapshot* of its state, and Raft throws away the log entries that the snapshot covers. A follower that is so far behind that the leader no longer has the entries it needs receives the leader's snapshot through `InstallSnapshot`.

APPENDIX B
REUSED CODE AND MODIFICATIONS

All the code we reused comes from the framework MIT's 2020 offering of the 6.824 course labs 2 and 3 [2]. Here is what was kept, what was changed, and what was self-written.

The starting point were the stubs in `raft.go`, `server.go`, `client.go` and `common.go`, which define the API that the tests call (`Make`, `Start`, `GetState`, `StartKVServer`, the `Clerk` methods and the RPC argument types). We kept that API and wrote everything behind it. Among the elements kept as they were, are the simulated in-memory network (`labrpc`), the gob wrapper (`labgob`), the Porcupine checker [3] with its key/value model, and the test suites with their harnesses (`test_test.go` and `config.go` in `src/raft` and `src/kvraft`).

Small changes to the labs' skeleton were made. The one-method `RPCClient` interface was added to `labrpc` and the test harnesses was made to use it instead of the concrete

`*labrpc.ClientEnd` type, which changes no test logic, but allowed tswitching between the in-memory simulated network of the tests and the actual TCP layer used in deployment. We also gave the `Persister` an optional file-backed mode (`MakeFilePersister`), used only in deployment, while the tests still use the in-memory version.

Everything else was self-written: the Raft implementation (about 1,000 lines in `raft.go`), the KV server and `Clerk`, the TCP transport (`labrpc/tcp.go`), the HTTP API with `/status` and `HTTPEnd` (`kvraft/http.go`), the node binary (`cmd/kvnode`), the load generator (`cmd/kvbench`), our TCP/HTTP test, the `Dockerfile` and `compose.yaml`, and the experiment scripts.

APPENDIX C
AUTOMATED TESTS: DETAILED DESCRIPTIONS

a) 2A tests (leader election): `InitialElection` checks that three nodes elect exactly one leader and agree on its term, and that the leader is still there two seconds later. `ReElection` disconnects the leader and expects a new one to be elected. The old leader coming back must not disturb the new one, with two of the three nodes disconnected no leader may be elected, and once a majority is back, one must be.

b) 2B tests (log replication): Here commands are submitted with `Start` and must be committed at the expected index by the expected number of nodes. `FailAgree` keeps committing with one follower disconnected and checks that it catches up when it comes back, while `FailNoAgree` checks that nothing commits when three of five followers are gone. In `Rejoin`, a disconnected leader accepts entries that can never commit, and when it reconnects it must drop them in favor of the new leader's log. `Backup` pushes this further: two minority partitions each pile up 50 uncommitted entries, and then nodes are reconnected in other combinations, so the leader has to overwrite long conflicting suffixes on its followers. `ConcurrentStarts` calls `Start` from several goroutines at once. Finally, `RPCBytes` and `Count` test for excessive communication, making sure that each command should be sent about once to each follower, an election may take at most 30 RPCs, and one idle second at most 60 RPCs.

c) 2C tests (persistence): The three `Persist` tests crash and restart nodes in different orders (all of them, only the leader, a follower while the others keep committing, a partitioned leader together with a follower), and a restarted node must come back with its term, vote and log. `Figure8` recreates the scenario behind the current-term commit rule: for 1000 iterations it gives a command to the leader and crashes it shortly after, usually before the entry commits, and restarts nodes whenever fewer than three are up. `Figure8Unreliable` does the same with disconnections instead of crashes, on an unreliable network, and turns on long reordering after 200 iterations. `UnreliableAgree` commits commands concurrently over the lossy network. The two `Churn` tests have three clients submitting commands while nodes are randomly disconnected, crashed and restarted every 700 ms; at the end, every value that a client saw committed must be in the final log of every node.

d) 3A tests (KV service): Most of these tests share the same routine, with five servers and three rounds of five seconds, in which each client appends to and reads its own key. Since each client is the only writer of its key, it knows what every `Get` must return, and any other value fails the test. After each round, every key is read back and each append must appear exactly once and in order, so both a lost write and a duplicate caused by a retry are caught. Options add more clients (five instead of one), an unreliable network, a partitioner that splits the servers into two random groups about once per second, and a restart of every server at the end of each round. `UnreliableOneKey` has five clients appending to the same key over the lossy network. `OnePartition` puts the leader on the minority side of a partition. The majority must keep serving requests, a `Put` and a `Get` sent to the minority must not complete, and the `Put` must go through once the partition heals. The `Get` on the minority side is the one that would catch an old leader serving stale reads.

e) 3B tests (snapshots): These tests set `maxraftstate` to 1000 bytes and, after each round, require the persisted Raft state to be at most eight times that, which means the log is really being trimmed. In `SnapshotRPC`, one server falls behind in a minority while the other two commit 50 writes and compact their logs. The partition then changes so that the lagging server is needed for a majority, and the only way it can catch up is through `InstallSnapshot`, since the leader has already thrown away the entries it is missing. `SnapshotSize` requires snapshots smaller than 500 bytes after 800 operations on a single key. The other six tests repeat the 3A combinations of faults, with up to 20 clients, with snapshots turned on.

f) 3A & 3B tests on linearizability: There are two *Linearizable* tests, one in 3A and one in 3B. Fifteen clients send random `Put`, `Append` and `Get` requests on shared keys to seven servers, with lossy network, partitions and restarts all active, and the start and end time of every operation is recorded. Porcupine [3] then searches for a single order of all the operations that respects real time (if one operation returned before another started, it must come first) and that explains every value returned by a `Get`. If no such order exists, the history is not linearizable and the test fails.

g) Our own test: All inherited tests use `labrpc` and the in-memory persister, so none of them runs the code that is used in the containers. To cover it, we wrote `TestHTTPAndTCPCluster`, which starts three nodes wired the same way as in the deployment: Raft over `net/rpc` on real TCP sockets, state saved to files on disk, and clients talking HTTP. It runs a `Put`, an `Append` and a `Get` through the `Clerk` and then checks three things: followers answer a write with HTTP 421 instead of applying it, `/status` reports the same leader on every node, and every node has $lastApplied \leq commitIndex \leq lastLogIndex$. This is only a smoke test without faults; the experiments of Section VI are what exercise the deployment under faults.

REFERENCES

[1] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*. Philadelphia, PA: USENIX Association, 2014, pp. 305–319. [Online]. Available: https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro

[2] R. Morris, F. Kaashoek *et al.*, "6.824 distributed systems: Labs 2 and 3," MIT Computer Science and Artificial Intelligence Laboratory, 2020, skeleton for `labrpc`, `labgob`, `porcupine` and the test suite reused in this work. [Online]. Available: https://pdos.csail.mit.edu/6.824/2020

[3] A. Athalye, "Porcupine: A fast linearizability checker for go," GitHub repository, 2017. [Online]. Available: https://github.com/anishathalye/porcupine

[4] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed. distributed-systems.net, 2023. [Online]. Available: https://www.distributed-systems.net/index.php/books/ds4/